



東華大學
DONGHUA UNIVERSITY

文本数据处理

DATA

计算机科学与技术学院

主讲教师：徐波、宋晖

7.1 文本数据

- 互联网发展形成的网络数据呈现爆发性增长
 - 数据80%以文本形式存放



- 如何从海量文档中快速发现并利用所需的知识？
 - 成为人工智能的热点研究应用方向

文本处理常见任务 (1)

- 任务1：文本分类

- 目标

- 按照一定的分类体系，将文档判别为预定的若干类中某一类或某几类

- 典型应用

- 垃圾邮件/短信分类
 - 新闻分类
 - 网页分类
 - 情感分析

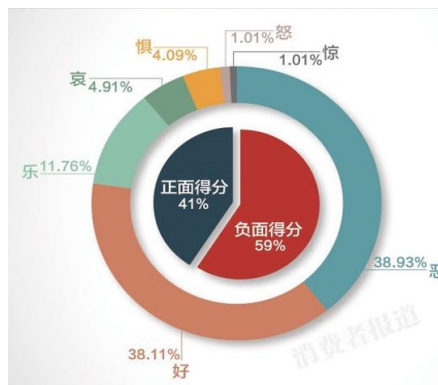
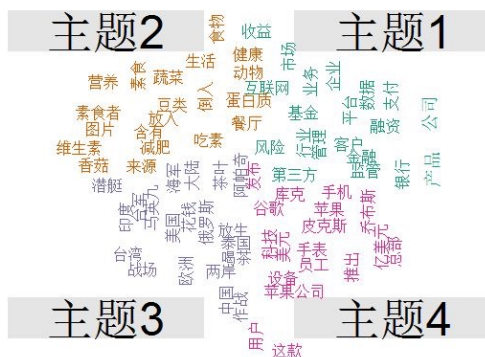
- 任务2：信息检索

- 目标

- 将信息（这里指代文本）按一定的方式组织起来，根据用户的需求将相关信息查找出来

- 典型应用

- 搜索引擎（谷歌、百度）



文本处理常见任务 (2)

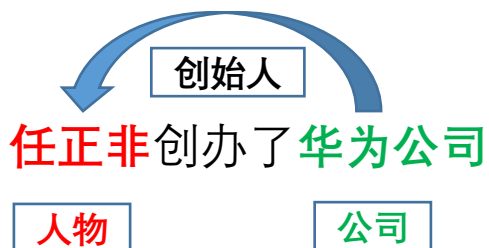
- 任务3：信息抽取

- 目标

- 将文本中包含的结构化或非结构化的信息抽取出来，组织成类似表格的形式

- 典型应用

- 命名实体识别
 - 关系抽取



- 任务4：自动问答

- 目标

- 用准确、简洁的自然语言回答用户以文本形式提出的问题

- 典型应用

- 问答机器人



文本处理常见任务 (3)

- 任务5：机器翻译

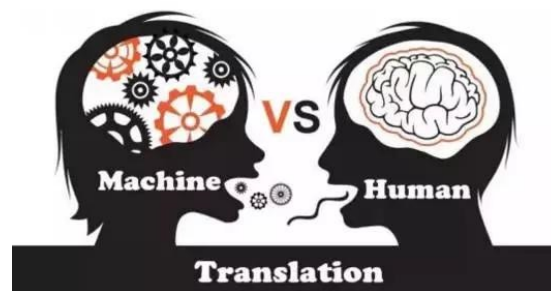
- 目标

- 一种自然语言文本自动转换为另一种自然语言文本

- 典型应用

- 翻译软件

- 谷歌翻译
 - 百度翻译
 - 有道翻译



- 任务6：自动摘要

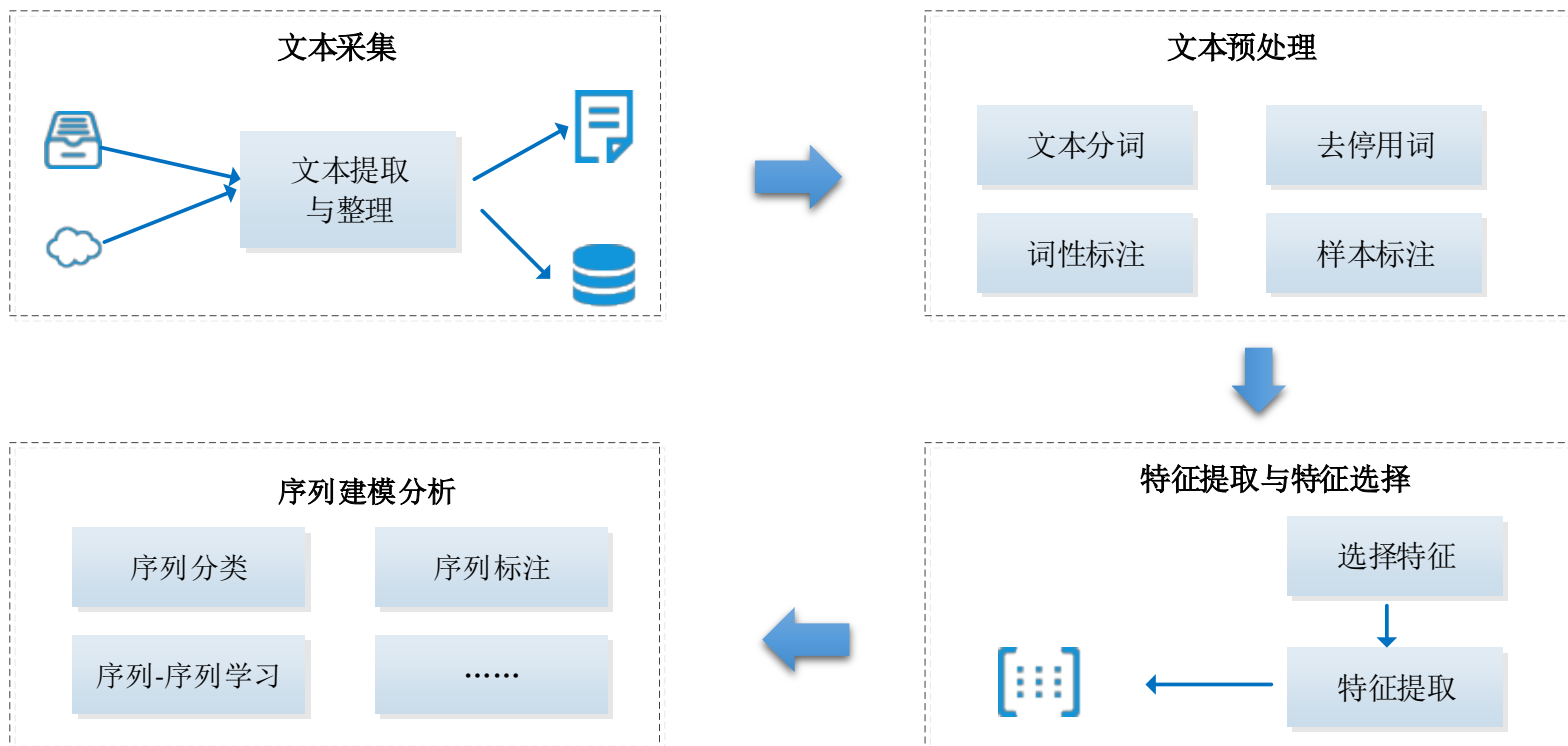
- 目标

- 从一份或多份文本中提取出来部分文字，它包含了原文本中的重要信息，且长度不超过或远少于原文本的一半

- 典型应用

- 搜索引擎

文本处理的基本步骤

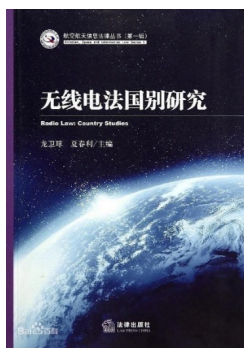


7.2 中文文本处理

- 英文文本处理工具相对比较成熟
 - 常用：Python的NLTK、工业级Spacy等
 - 提供多项自然语言的基本处理功能
 - 词性分析、语法分析、实体识别、依赖分析、文本分类等
- 中文语言学特性与英文差别大，处理难度更大
 - 分词工具：Ansj、Jieba
 - 中文处理包：THULAC（清华大学）、FNLP（复旦）、哈工大LTP、HanLP（大快公司）

中文分词

- 词：最小的能够独立活动的有意义的语言成分
 - 英文单词之间以**空格**分界
 - 汉语以**字**为基本书写单位，词语之间**没有明确区分**
- 分词（Chinese Word Segmentation）
 - 将连续的字序列按照一定的规范重新组合成词序列



无线电|法国|别|研究

无线电|法|国别|研究

Python 的中文分词工具包

- 中文分词工具包
 - jieba、SnowNLP、THULAC、NLPIR
- Python分词库：jieba

例句：“如何成为一名优秀的数据分析师”

精确模式

'如何', '成为', '一名', '优秀',
'的', '数据', '分析师'

试图将句子最精确地切开，
适用于文本分析任务

全模式

'如何', '成为', '一名', '名优', '优
秀', '的', '数据', '数据分析', '分析',
'分析师'

找出所有可以成词的词语，速度
块，但不能解决歧义

搜索模式

'如何', '成为', '一名', '名优',
'优秀', '的', '数据', '数据分
析', '分析', '分析师'

精确模式基础上，对长词再切分，
适合用于搜索引擎分词

jieba中文分词函数

函数	参数说明
<code>cut(sentence, cut_all=False, HMM=True)</code>	<code>sentence</code> ：待分词的字符串； <code>cut_all</code> ：是否采用全模式； <code>HMM</code> ：是否使用 HMM 模型。返回可迭代的 generator。支持精确模式和全模式
<code>cut_for_search(sentence, HMM=True)</code>	<code>sentence</code> ：待分词的字符串； <code>HMM</code> ：是否使用 HMM 模型。返回可迭代的 generator。该方法为搜索引擎模式，适合用于建立搜索引擎构建倒排索引，粒度比较细
<code>lcut(sentence, cut_all=False, HMM=True)</code>	与cut函数类似，直接返回词列表
<code>lcut_for_search(sentence, HMM=True)</code>	与cut_for_search函数类似，直接返回词列表

扩展：jieba分词 下载地址<https://pypi.org/project/jieba/>

分词实现

【例子7-1】 将文本句子 “如何成为一名优秀的数据分析师” 进行分词

```
>>> import jieba
```

```
>>> jieba.lcut("如何成为一名优秀的数据分析师")
```

```
['如何', '成为', '一名', '优秀', '的', '数据', '分析师']
```

```
>>> jieba.lcut("如何成为一名优秀的数据分析师", cut_all = True)
```

```
['如何', '成为', '一名', '名优', '优秀', '的', '数据', '数据分析', '分析', '分析师']
```

```
>>> jieba.lcut_for_search("如何成为一名优秀的数据分析师")
```

```
['如何', '成为', '一名', '优秀', '的', '数据', '分析', '分析师']
```

词性标注(POS)

- 词性 (part-of-speech)
 - 如名词、动词、形容词、代词等
- 词性标记集合 (中科院计算所ictclas)
- jieba的posseg模块提供词性标
 - cut()和lcut()函数分词, 同时标注词性

n 名词

nr 人名

nr1 汉语姓氏

nr2 汉语名字

nrj 日语人名

nrfs 音译人名

ns 地名

nsf 音译地名

nt 机构团体名

nz 其它专名

nl 名词性惯用语

ng 名词性语素

```
>>> import jieba.posseg as pseg
>>> words = pseg.cut("如何成为一名优秀的数据分析师 ")
>>> for word, tag in words:
    print( 'word: {}, tag: {}'.format(word, tag) )
```

word:如何, tag:r
word:成为, tag:v
word:一名, tag:m
word:优秀, tag:a
word:的, tag:uj
word:数据, tag:n
word:分析师, tag:n

7.2.3 文本特征提取

- 文本内容转换成数字特征向量
 - 词袋模型 (Bag of word)
 - TF-IDF

文档集

句子1: “我是中国人, 我爱中国”

句子2: “我是上海人”

句子3: “我住在上海松江大学城”

上海	中国	人	住	在	大学城	我	是	松江	爱
0	1	2	3	4	5	6	7	8	9

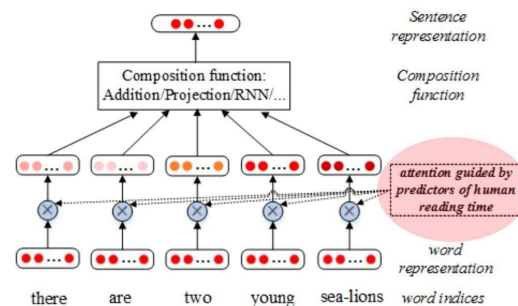
[0 2 1 0 0 0 2 1 0 1]

[1 0 1 0 0 0 1 1 0 0]

[1 0 0 1 1 1 1 0 1 0]

词向量模型

index	word	Real value vector(dim=50)
1	I	[0.12, 0.31, ..., 1.2, 0.92]
2	Like	[0.12, 0.31, ..., 1.2, 0.92]
3	This	[0.12, 0.31, ..., 1.2, 0.92]
4	Movie	[0.12, 0.31, ..., 1.2, 0.92]
5	Is	[0.12, 0.31, ..., 1.2, 0.92]
6	So	[0.12, 0.31, ..., 1.2, 0.92]
7	Bad	[0.12, 0.31, ..., 1.2, 0.92]



词袋模型

- 将一条文本仅看作一些独立的词语的集合，忽略文本的词序、语法和句法
- 词袋模型统计在一个句子中每个单词出现的次数

文档集

句子1: “我是中国人, 我爱中国”

句子2: “我是上海人”

句子3: “我住在上海松江大学城”

字典

上海	中国	人	住	在	大学城	我	是	松江	爱
0	1	2	3	4	5	6	7	8	9

$[0 \quad \mathbf{2} \quad 1 \quad 0 \quad 0 \quad 0 \quad 2 \quad 1 \quad 0 \quad 1]$

$$[1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0]$$
$$[1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0]$$

向量

词袋模型构建过程

- 构建过程分3步

1. 对文档集中的所有文本进行分词
2. 构建整个文档集的词典，假设词典长度为 n
3. 为每条文本生成长度为 n 的一维向量，向量中的每一维的值为字典中对应序号的词在该文本中出现的次数

步骤二：构造字典

{'上海':0, '中国':1, '人':2, '住':3, '在':4, '大学城':5, '我':6, '是':7, '松江':8, '爱':9}

步骤一：分词

句子1：“我/是/中国/人/， /我/爱/中国”

句子2：“我/是/上海/人”

句子3：“我/住/在/上海/松江/大学城”

步骤三：生成向量

句子1: [0 2 1 0 0 0 2 1 0 1]

句子2: [1 0 1 0 0 0 1 1 0 0]

句子3: [1 0 0 1 1 1 1 0 1 0]

词袋模型实现

- Scikit-learn的文本特征提取
 - feature_extraction.text模块的CountVectorizer类

词袋模型初始化: **cv = CountVectorizer(token_pattern)**

生成词袋向量: **cv_fit = cv.fit_transform(split_corpus)**

参数说明：	
token_pattern	token模式的正则表达式，默认为字符数两个及以上的token
split_corpus	文本词列表

文本词袋特征生成

- 使用Scikit-learn的`feature_extraction.text`实现句子向量化

```
from sklearn.feature_extraction.text
import CountVectorizerimport jieba
#给出文档集，放在字符串列表中
corpus = [
    "我是中国人，我爱中国",
    "我是上海人",
    "我住在上海松江大学城"
]
split_corpus = []

#初始化分词结果的列表#循环为corpus中的每个字符串分词
for c in corpus:
    #将jieba分词后的字符串列表拼接为一个字符串，元素之间用" "分割
    s = " ".join(jieba.lcut(c)) #将分词得到的列表
    split_corpus.append(s) #将分词结果字符串添加到列表中
print(split_corpus)

#生成词袋
cv = CountVectorizer()
cv_fit=cv.fit_transform(split_corpus)
print(cv.get_feature_names()) #显示特征列表
print(cv_fit.toarray()) #显示特征向量
```

特征列表:

['上海', '中国', '大学城', '松江']

特征向量:

```
[[0 2 0 0]
 [1 0 0 0]
 [1 0 1 1]]
```

缺省：字符数>1为特征

#修改token_pattern默认参数：保留每个词

```
cv =CountVectorizer(token_pattern=r"(?u)\b\w+\b")
```

TF-IDF

- “词频-逆文本频率”，用于评估某词对于一篇文档的重要程度
 - TF: 词频
 - 如果某个词在某文档中出现多次，则说明这个词可能比较重要或者是文档常用词

$$TF = \frac{\text{该词在文档中出现的次数}}{\text{文档中总词数}}$$

- IDF: 逆文档频率
 - 主要用来去除文档常用词，如停用词等的IDF值就会很低

$$IDF = \log \left(\frac{\text{文档集中文档总数}}{\text{文档集中包含该词的文档数}} \right)$$

- TF-IDF : $TF \times IDF$
 - 倾向于过滤掉常见的词语，保留重要的词语

文档集	上海	中国	人	住	在	大学城	我	是	松江	爱
句子1: “我是中国人，我爱中国”	0.	0.73	0.28	0.	0.	0.	0.43	0.28	0.	0.36]
句子2: “我是上海人”	0.53	0.	0.53	0.	0.	0.	0.41	0.53	0.	0.]
句子3: “我住在上海松江大学城”	0.34	0.	0.	0.45	0.45	0.45	0.27	0.	0.45	0.]

TF-IDF计算

- Scikit-learn中计算TF-IDF 值， 两种方法
 1. CountVectorizer类向量化之后再调用TfidfTransformer类

使用feature_extraction.text模块的TfidfTransformer类

```
#在例7-3的代码后添加以下代码。  
from sklearn.feature_extraction.text import TfidfTransformer  
#将文本词袋特征表示转化为TF-IDF特征。  
tfidf_transformer = TfidfTransformer()  
tfidf_fit = tfidf_transformer.fit_transform(cv_fit)  
print(tfidf_fit.toarray())      #显示TF-IDF特征向量
```

特征表示

```
[[ 0.          0.72777291  0.27674503  0.          0.          0.  
  0.42983441  0.27674503  0.          0.36388646]  
[ 0.52682017  0.          0.52682017  0.          0.          0.  
  0.40912286  0.52682017  0.          0.          ]  
[ 0.34261996  0.          0.          0.45050407  0.45050407  0.45050407  
  0.26607496  0.          0.45050407  0.          ]
```

- Scikit-learn中计算TF-IDF 值，两种方法

- 2. 用TfidfVectorizer完成向量化与TF-IDF计算

直接使用feature_extraction.text模块的TfidfVectorizer类计算

```
from sklearn.feature_extraction.text import TfidfVectorizer
#直接用分词后得到的列表计算TF-IDF特征表示
tfidf = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
tfidf_fit=tfidf.fit_transform(split_corpus)
print(tfidf_fit.toarray()) #显示TF-IDF特征向量
```

文档集中3条文本的TF-IDF特征表示

```
[[ 0.    0.72777291  0.27674503  0.    0.    0.    0.42983441  0.27674503  0.    0.36388646]
 [ 0.52682017  0.    0.52682017  0.    0.    0.    0.40912286  0.52682017  0.    0.    ]
 [ 0.34261996  0.    0.    0.45050407  0.45050407  0.45050407  0.26607496  0.    0.45050407  0.    ]]
```

两种方法得到的TF-IDF向量
一致

词向量模型构建过程

- 构建过程分3步

1. 对文档集中的所有文本进行分词
2. 构建整个文档集的词典，每个词用一个m维向量表示 ($m \ll n$)
3. 为每条文本生成一个二维数组，数组中的每一维的值为字典中对应的词的一维向量

步骤二：构造词向量字典

上海	中国	人	住	在	大学城	我	是	松江	爱
[0.1,0.2,0.3]	[0.8,0.3,0.1]	[0.6,0.4,0.2]	[0.8,0.9,0.2]	[0.4,0.7,0.1]	[0.6,0.6,0.2]	[0.6,0.1,0.8]	[0.1,0.9,0.1]	[0.7,0.7,0.7]	[0.4,0.5,0.6]

步骤一：分词

句子1：“我/是/中国/人/，/我/爱/中国”

句子2：“我/是/上海/人”

句子3：“我/住/在/上海/松江/大学城”

步骤三：生成向量

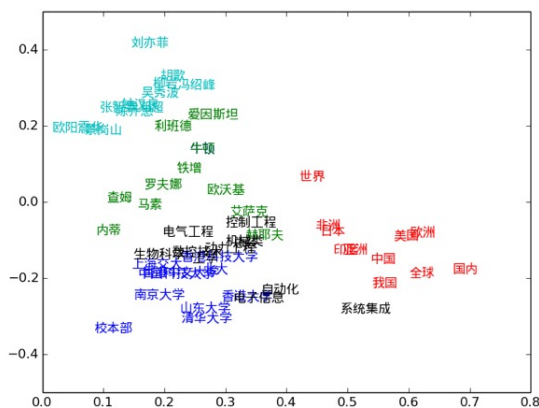
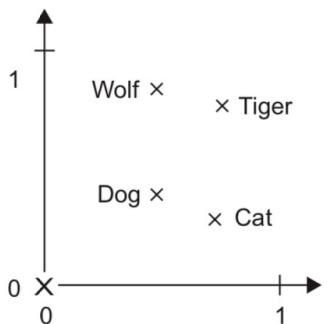
句子1：[[0.6,0.1,0.8],[0.1,0.9,0.1], [0.8,0.3,0.1], [0.6,0.4,0.2],...]

句子2：[[0.6,0.1,0.8],[0.1,0.9,0.1], [0.1,0.2,0.3], [0.6,0.4,0.2]]

句子3：[[0.6,0.1,0.8], [0.8,0.9,0.2], [0.4,0.7,0.1], [0.1,0.2,0.3],...]

使用Embedding层学习词嵌入

- 希望词向量之间的几何关系能够反映词的语义关系



- 目前没有理想的词嵌入空间，适用所有的自然语言问题
 - 电影评论情感分析 vs. 法律文档问答 语义存在区别
 - 为每个新任务学习一个新的嵌入空间，使用深度神经网络学习获得一个较好的词嵌入
- 任务可用的训练数据很少，无法学习适合的词嵌入
 - 加载预训练的词嵌入向量

思考与练习

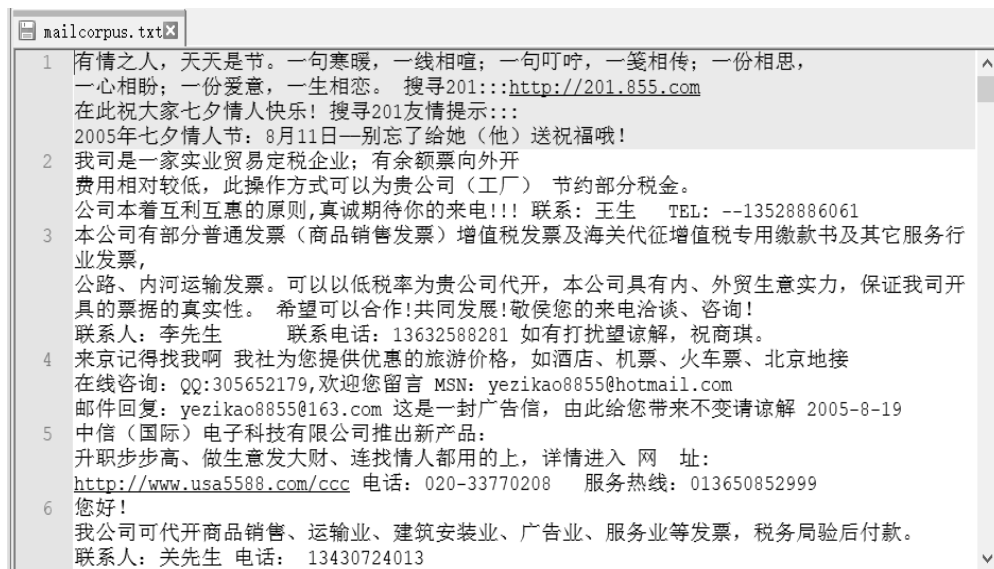
- 1.在例6-3的文档集中添加2条文本，“松江大学城有很多大学”、“大学城共有15万余大学生”。计算文档集中每条文本的词袋和TF-IDF特征表示。
- 2.比较练习1的词向量与例题中的词向量是否一致，说明原因。

7.3 实例：垃圾邮件识别

- 识别垃圾邮件的技术
 - 关键词识别
 - IP黑白名单
 - 反向DNS查找
 - 意图分析技术链接URL
 - 分类算法等（最为常见）
- 分类算法
 - 收集大量的垃圾邮件和非垃圾邮件
 - 建立垃圾邮件库和非垃圾邮件库
 - 提取其中的特征
 - 训练分类模型

邮件分类：正文文本特征

- 数据来源 —Trec06C数据集
 - 包含64,620封邮件，正常：21,766封，垃圾：42,854封
 - 每封邮件保存为单独文件
 - index文件保存所有邮件的类别（spam or ham）
- 随机选取：5000封垃圾邮件和5000封正常邮件
- 预处理
 - 提取邮件正文，去掉换行符、多余空格等
 - 1封邮件文本1行，保存为mailcorpus.txt文件



mailcorpus.txt

```
1 有情之人，天天是节。一句寒暖，一线相喧；一句叮咛，一笺相传；一份相思，
  一心相盼；一份爱恋，一生相恋。 搜寻201:::http://201.855.com
  在此祝大家七夕情人快乐！搜寻201友情提示:::
  2005年七夕情人节：8月11日—别忘了给她（他）送祝福哦！
2 我司是一家实业贸易定税企业；有余额票向外开
  费用相对较低，此操作方式可以为贵公司（工厂）节约部分税金。
  公司本着互利互惠的原则，真诚期待你的来电!!! 联系：王生 TEL: --13528886061
3 本公司有部分普通发票（商品销售发票）增值税发票及海关代征增值税专用缴款书及其它服务行
  业发票，
  公路、内河运输发票。可以以低税率为贵公司代开，本公司具有内、外贸生意实力，保证我司开
  具的票据的真实性。 希望可以合作!共同发展!敬候您的来电洽谈、咨询！
  联系人：李先生 联系电话：13632588281 如有打扰望谅解，祝商祺。
4 来京记得找我啊 我社为您提供优惠的旅游价格，如酒店、机票、火车票、北京地接
  在线咨询：QQ:305652179,欢迎您留言 MSN: yezikao8855@hotmail.com
  邮件回复：yezikao8855@163.com 这是一封广告信，由此给您带来不便请谅解 2005-8-19
5 中信（国际）电子科技有限公司推出新产品：
  升职步步高、做生意发大财、寻找情人都用的上，详情进入 网 址：
  http://www.usa5588.com/ccc 电话：020-33770208 服务热线：013650852999
6 您好！
  我公司可代开商品销售、运输业、建筑安装业、广告业、服务业等发票，税务局验后付款。
  联系人：关先生 电话： 13430724013
```

构建文本分类特征训练集

- 使用词袋模型或TF-IDF模型提取特征
 - 得到 $m \times n$ 的矩阵 X
 - m 为10,000, n 为文本集的字典词条数目)
 - 标签向量 y 长度为 m , 元素值为0或1。

```
import jieba
from sklearn.feature_extraction.text import CountVectorizer

#从文本读取文本，放在列表中
train_file = open("data/train.txt", 'r', encoding = "utf-8")
corpus = train_file.readlines() #列表中的每个元素为一行文本
#分词
split_corpus = []
for c in corpus:
    split_corpus.append(" ".join(jieba.lcut(c)))
#使用词袋模型提取特征，得到文本特征矩阵
cv = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X = cv.fit_transform(split_corpus).toarray()
#构造标签向量，垃圾标签为0，正常标签为1
y = [0] * 5000 + [1] * 5000
```

模型训练和验证

- 将数据集随机切分为训练集和测试集（40%）
- 使用SVM进行训练

```
from sklearn import model_selection
from sklearn import svm
from sklearn import metrics
#将特征集 分为训练集和测试集
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.4,
random_state = 0)

#使用SVM训练分类模型
svm = svm.SVC(kernel='rbf', gamma=0.7, C = 1.0)
svm.fit(X_train, y_train)

#SVM分类性能
y_pred_svm = svm.predict(X_test)

print("SVM accuracy:\n",svm.score(X_test, y_test))
print("SVM report:\n",metrics.classification_report(y_test, y_pred_svm))
print("SVM matrix:\n",metrics.confusion_matrix(y_test, y_pred_svm))
```

svm训练文本数据需要的时间较长，可以换用其他分类模型，如朴素贝叶斯（sklearn.naive_bayes.NB）训练时间将极大缩短，而且能取得更好的垃圾邮件的识别效果。

	precision	recall	f1-score	support
0	1.00	0.85	0.92	1993
1	0.87	1.00	0.93	2007
avg / total	0.93	0.93	0.92	4000

思考与练习

- 1.将邮件特征提取从词袋模型改为TF-IDF模型，比较使用不同的特征计算模型的分类性能。
- 2.使用Scikit-learn的CountVectorizer()函数初始化词袋模型时，设置不同的特征个数生成邮件的特征表示向量，比较训练分类模型所耗费的时间，以及分类模型分类的准确性。特征个数越多是否意味分类性能越好？

综合练习

Trec06C数据集每封邮件包含发件人、收件人、标题、正文以及附件等完整信息。本章6.3中只使用了邮件的正文的文本特征训练垃圾邮件分类器。考虑将发件人、收件人以及标题等特征也转化为向量数据，添加到文本向量中，训练邮件分类器，并与只使用正文的分类器进行性能比较。